CHAPTER 4

More on React Native Components

Content

- React native Image and ImageBackground
- React native Scroll View and RefreshControl
- React native List View
  - FlatList and SectionList
- ActivityIndicator and ProgressBar
- React native Web View
- React native Picker
- React native Status Bar

Core Component – Image

- **Images** are an essential aspect of any mobile app. This component enable use to displays different types of images.
- **Adding Image**
  - To add images in the application, you first need to import the **Image** component from **react-native**.

```
import { Image } from 'react-native‘
```

- **Syntax:**

  <Image/>
- **Image Styles**
  - width, height, borderRadius, borderColor, borderWidth, . .
- **Image props**
  - **source**: The image source (either a remote URL or a local file resource).

- The React Native image component allows you to display images from different sources, such as:
  - Static resources
  - Network images
  - **Temporary local images and**
  - **Local disk images, i.e. from the camera roll**

- **Display Static Images**
  - To display a static image, the first thing to add is the image file in the **assets** folder of the project.
  - **Static images** are loaded by providing the image path.
  - The code for displaying a static image will look something like this:

```
import { StatusBar } from "expo-status-bar";
import React from "react";
import { StyleSheet, Text, View, Image} from "react-native";


export default function App() {
  return (
    <View style={styles.container}>
        <StatusBar style="auto" />
        <Text>.......................</Text>

        <Image
          style={{ width: 100, height: 100, marginBottom: 15 }}
          source={require("./assets/facebook.png")}
        />

    </View>
  );
}
```

- **Network Images**
  - Displaying an image from a network or web-based source is similar to displaying a static image.
    - Within the **Image** component, use the **source** attribute and set the path of the image in an object with the **uri** key, as shown below.
    - Don't forget that for network and URI-encoded images, you will need to **manually specify the dimensions of your image.**

```
import { StatusBar } from "expo-status-bar";
import React from "react";
import { StyleSheet, Text, View, Image } from "react-nati


export default function App() {
  return (
    <View style={styles.container}>

        <StatusBar style="auto" />

        <Text>....................</Text>

        <Image
          style={{ width: 100, height: 100 }}
          source={{ uri: "https://reactjs.org/logo-og.png'
        />

    </View>
  );
}
```

- **URI Data Images**
  - You can also display images via a data URI, in which case all the image data is actually encoded in the URI.
    - This is only recommended for very small or dynamic images.
  - For a URI-encoded image, you'll provide the image data with a source attribute like

```
const encodedBase64 = 'R0lGODlhAQABAIAAAAAA...7‘

<Image source={{ uri: `${encodedBase64}`}} />
```

Core Component – ImageBackground

- **Background Images**
  - You can also use an image as the background for your screen.
  - To do so, get a background image of your choice and add it to the assets folder.
  - Next, import the **ImageBackground** component from react-native as shown below.
    import {ImageBackground } from 'react-native';

- The **ImageBackground** component wraps and displays a background for whatever elements are inside it.
  - In this case, we will replace the View tag with the **ImageBackground** tag and wrap it around all the content in the app.
- **ImageBackground styles**
  - width, height, padding, margin, . . .

```
return (

    <ImageBackground
      source={require("./assets/back.jpeg")}
      style={styles.back_image}
    >
      <Text>.......................</Text>
      <StatusBar style="auto" />
      <Image
        style={{ width: 100, height: 100, marginBottom:
        source={require("./assets/facebook.png")}
      />
      <Text>.............</Text>

      <Image
        style={{ width: 100, height: 100 }}
        source={{ uri: "https://reactjs.org/logo-og.png'
      />
    </ImageBackground>

);
```

Core Component – ScrollView

- The **ScrollView** is a generic scrolling container that can contain multiple components and views.
- The scrollable items can be heterogeneous, and you can scroll both vertically and horizontally (by setting the **horizontal** property).

- The ScrollView works best to present a small amount of things of a limited size.
  - All the elements and views of a ScrollView are **rendered**, **even if they are not currently shown on the screen**.
  - If you have a long list of items which cannot fit on the screen, you should use a **FlatList** instead.

- **ScrollView props**
  - **horizontal:** When true, the scroll view's children are arranged horizontally in a row instead of vertically in a column.
  - **scrollEnabled**: When false, the view cannot be scrolled via touch interaction. Note that the view can always be scrolled by calling **scrollTo**.
  - **pagingEnabled**: When true, the scroll view stops on multiples of the scroll view's size when scrolling. This can be used for horizontal pagination
  - **refreshControl**: A **RefreshControl** component, used to provide pull-to-refresh functionality for the ScrollView. Only works for vertical ScrollViews (horizontal prop must be false).

```
import React from 'react';
import { Image, ScrollView, Text } from 'react-native';

const logo = {
  uri: 'https://reactnative.dev/img/tiny_logo.png',
  width: 64,   height: 64
};
const App = () => (
  <ScrollView>
    <Text style={{ fontSize: 30 }}>Scroll me plz</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{ fontSize: 30 }}>If you like</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{ fontSize: 30 }}>Scrolling down</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{ fontSize: 30 }}>What's the best</Text>
    <Image source={logo} />
    <Image source={logo} />
    <Text style={{ fontSize: 30 }}>React Native</Text>
  </ScrollView>
);

export default App;
```

Core Component – RefreshControl

- This component is used inside a **ScrollView** or **ListView** to add pull to refresh functionality.
  - When the ScrollView is at scrollY: 0, swiping do

```
<RefreshControl
    refreshing={true}
    onRefresh={()=>{}}
    colors={['#ff00ff']}
/>
```

- **RefreshControl props**
  - **refreshing:** Whether the view should be indicating an active refresh. (**required** props): [boolean value]
  - **colors (**Android): The colors (at least one) that will be used to draw the refresh indicator. [array of colors]
    - **tintColor (**iOS): The color of the refresh indicator.
  - **onRefresh:** Called when the view starts refreshing. [function]

```
<ScrollView
    refreshControl={
        <RefreshControl
            refreshing={Refreshing}
            onRefresh={onRefresh}
            colors={['#ff00ff']}
        />
    }
>
```

React native ListView

- React Native provides a suite of components for presenting lists of data.
- Generally, we'll use either
  - FlatList or
  - SectionList.

Core Component – FlatList

- The **FlatList** component displays the similar structured data in a **scrollable** list.
  - It works well for **large lists of data** where the number of list items might change over time.
  - The **FlatList shows only those renders elements** which are currently displaying on the screen, not all the elements of the list at once.
- The **FlatList** component takes **two required props**:

1. **data**: is the source of elements for the list,
2. **renderItem:** takes one item from the source and returns a formatted component to render

- **Import:**
  - import {**FlatLIst**} from ‘react-native’
- **Syntax:**
  - **<FlatList />**
- **FlatList props**
  - **data**: For simplicity, data is a plain array. If you want to use something else, like an immutable list
  - **renderItem**: takes an item from data and renders it into the list. Provides additional metadata like index if you need it.

- **horizontal:** If true, renders items next to each other horizontally instead of stacked vertically.
- **inverted:** Reverses the direction of scroll. Uses scale transforms of -1
- **keyExtractor:** Used to extract a unique key for a given item at the specified index. Key is used for caching and as the react key to track item re-ordering. The default extractor checks item.key, then item.id, and then falls back to using the index, like React does.
  - **(item: object, index: number) => string;**
- **refreshControl**: A **RefreshControl** component, used to provide pull-to-refresh functionality for the ScrollView. Only works for vertical ScrollViews (horizontal prop must be false).

FlatList features support

- Fully cross-platform.
- Optional horizontal mode.
- Configurable viewability callbacks.
- Header support.
- Footer support.
- Separator support.
- Pull to Refresh.
- Scroll loading.
- ScrollToIndex support.
- Multiple column support.

```react
<FlatList
    style={styles.body}
    // horizontal={false}
    // numColumns={2}
    // inverted={false}
    keyExtractor={(item, index)=>index.toString()}
    data={Items}
    renderItem={({item})=>(
        <View style={styles.item}>
            <Text style={styles.text}>{item.name}</Text>
        </View>
    )}
    refreshControl={
        <RefreshControl
            refreshing={Refreshing}
            onRefresh={onRefresh}
            colors={['#ff00ff']}
        />
    }
/>
```

```react
const [Items, setItems]=useState([
    {name:'Item 1'},
    {name:'Item 2'},
    {name:'Item 3'},
]);
```

```react
const [Refreshing, setRefreshing]=useState(false);
const onRefresh=()=>{
    setRefreshing(true);
    setItems([...Items, {name:'Item 105'}]);
    setRefreshing(false);
}
```

Core Component – SectionList

- As the name suggests, **SectionList** enables you to show a **sectioned list in your UI**.
  - It is a performant interface for rendering sectioned lists with useful features, including
    - Fully cross-platform.
    - Configurable viewability callbacks.
    - List header support.
    - List footer support.
    - Item separator support.
    - Section header support.
    - Section separator support.
    - Heterogeneous data and item rendering support.
    - Pull to Refresh.
    - Scroll loading.

- **Import:**
  - import {**SectionLIst**} from ‘react-native’
- **Syntax:**
  - **<SectionList/>**
- **SectionList props**
  - **sections:** The actual data to render, akin to the data prop in FlatList. **[Required]**
  - **renderItem**: Default renderer for every item in every section. Can be over-ridden on a per-section basis. Should return a React element. **[Required]**

- **renderSectionHeader:** Rendered at the top of each section.
  - These stick to the top of the ScrollView by default on iOS.
- **keyExtractor:** Used to extract a unique key for a given item at the specified index. Key is used for caching and as the React key to track item re-ordering. The default extractor checks item.key, then falls back to using the index, like React does. Note that this sets keys for each item, but each overall section still needs its own key.
- **stickySectionHeadersEnabled:** Makes section headers stick to the top of the screen until the next one pushes it off. Only enabled by default on iOS because that is the platform standard there

```
<SectionList
    keyExtractor={(item,index)=>index.toString()}
    sections={taskData}
    renderItem={({item})=>(
        <Text style={styles.text}>{item}</Text>
    )}
    renderSectionHeader={({section})=>(
        <View style={styles.item}>
            <Text style={styles.text}>{section.title}</Text>
        </View>
      )}
    stickySectionHeadersEnabled={true}
/>
```

```
const taskData=[
    {
        title:'New Task',
        data:['Buy groceries','Feed Cat','Sleep for 3 hours','Water Plants']
    },
    {
        title:'Completed Task',
        data:['Drink Water','Make a section list note','Share this note']
    }
]
```

Core Component – ActivityIndicator

- **ActivityIndicator** is a component for displaying loading action . It is the same as the circular loader / Progress Bar.
  - It is used to show the progress of long-running task so that the user can understand something is in progress.
- **Import:**
  - import {**ActivityIndicator**} from ‘react-native’
- **Syntax:**
  - **<ActivityIndicator/>**

- **ActivityIndicator props**
  - **animating**: Option to show the indicator (by default it is true) or hide it (false).
  - **size**: set the size of indicator ('small', 'large', number). The default size is small. Number format support only in android.
  - **color**: set the foreground color of the spinner (default is gray).
  - **hidesWhenStopped**: It provides an option to show or hide the indicator when there is no animating (true by default). (ios only)

```
<ActivityIndicator
    animating = {true}
    size="small"
    color="#44ff00"
/>
<ActivityIndicator
    animating = {true}
    size="large"
    color="red"
/>
```

Core Component – ProgressBar

- The **ProgressBarAndroid** component of **React Native** is used to indicate the progress status of some activity or app loading something.
  - **ProgressBarAndroid** component works only on Android platform. To use the progress bar in iOS platform, a **ProgressViewIOS** component is used.
- **Import:**
  - import {**ProgressBarAndroid, ProgressViewIOS** } from ‘react-native’
- **Syntax:**
  - **< ProgressBarAndroid />**

- **ProgressBar props**
  - **animating** (bool) : It is used to show or hide the progress bar. Its default value is true to show, (false to hide).
  - **color (**color): It sets the color of progress bar.
  - **Indeterminate** (indeterminateType): It shows the intermediate progress status of progress bar. It can only be false if the styleAtte of progress bar is Horizontal.
  - **progress** (number): It is a progress value between 0 and 1.
  - **styleAttr** (enum): It sets the style of progress bar. There are various style of progress bar in React Native such as Horizontal, Normal (default), Small, Large, Inverse, SmallInverse, and LargeInverse.

```
<Text style = {{fontSize: 20, color: '#000'}}> Progress Value:
{ parseFloat((this.state.progressStatus * 100).toFixed(3))} %</Text>{
    ( Platform.OS === 'android' )
    ?
      ( <ProgressBarAndroid
        styleAttr = "Horizontal"
        progress = { this.state.progressStatus }
        indeterminate = { false }
        /> )
    :
      ( <ProgressViewIOS
        progress = { this.state.progressStatus }
        /> )
}
```

Core Component – WebView

- React Native **WebView** is a component which is used to **load web content or web page.**
  - The WebView component is imports from core react-native library.
    - ~~**Deprecated component**~~
  - Now, the WebView is replaced from the built-in core react-native, and placed in **react-native-webview** library.
    - **Note:** The React Native WebView recommended to import react-native-webview or react-native-community library.

- **Installed package**
  - npm start **react-native-webview**
- **Import:**
  - import {**WebView**} from ‘react-native-webview’
- **Syntax:**
  - **<WebView/>**
- **WebView props**
  - **source**: Loads static HTML or a URI (with optional headers) in the WebView.
    - Load uri : **uri (string)** - The URI to load in the WebView
    - Static HTML: **html (string)** - A static HTML page to display in the WebView.

- **Types of WebView contents:**
  - **Display HTML code as a string:** The HTML string code is passed into **html** prop inside **source** property.

```
import React from 'react';
import { WebView } from 'react-native-webview';
export default function MyWebView() {
  return (
      <WebView source={{ html: '<h1>Hello Ethiopia</h1>' }} style={{marginTop:20}}/>
  );
}
```

- **Display the internal web page:** Create an internal web page inside a directory and pass its full path in source property.

```
import React from 'react';
import { WebView } from 'react-native-webview';
export default function MyWebView() {
  return (
      <WebView source={require("./index.html")} style={{marginTop:20}}/>
  );
}
```

- **Display the remote web page:** A remote web page is loaded using **uri** tag with **source** property.

```
import React from 'react';
import { WebView } from 'react-native-webview';
export default function MyWebView() {
  return (
      <WebView source={{ uri: 'https://gmail.com' }} style={{marginTop:20}}/>
  );
}
```

Core Component – Picker

- **React Native Picker** is component which is used to select an item from the multiple choices.
  - This is the same as a Dropdown option.
- **Picker** is used when we need to provide an alternative to choose from multiple options.
  - It is used by importing the **Picker** component from the react-native library.
- **Import:**
  - import {**Picker**} from ‘react-native’

- **Syntax:**

```
<Picker>
    <Picker.Item label=‘’ value=‘’>
</Picker>
```

- **Picker props**
  - **style**: It is a picket style type. (height and weight)
  - **selectedValue**: Returns the selected value.
  - **onValueChange**(itemValue, itemPosition): It is a callback prop and called when an item is selected. It takes two parameters (itemValue: the item (value) which is selected, itemPosition: the position (index) of a selected item).

```
import React,{useState} from 'react';
import {StyleSheet,View, Text,Picker} from 'react-native'
export default function MyPicker() {
    const [selectedValue, setSelectedValue] = useState("js");
    return (
      <View style={styles.container}>
        <Picker
          selectedValue={selectedValue}
          style={{ height: 50, width: 150 }}
          onValueChange={(itemValue, itemIndex) => setSelectedValue(itemValue)}
        >
          <Picker.Item label="Java" value="java" />
          <Picker.Item label="JavaScript" value="js" />
          <Picker.Item label="C++" value="cpp" />
        </Picker>
      </View>
    );
}
```

Core Component – StatusBar

- **React Native StatusBar** is a component which is used to decorate status bar of the app.
  - It is used by importing the **StatusBar** component from the react-native library.
  - We can use multiple StatusBar at the same time.
- **Import:**
  - import {**StatusBar**} from ‘react-native’
- **Syntax:**
  - **<StatusBar/>**

- **StatusBar props**
  - **backgroundColor**: It sets the background color of the status bar.
  - **translucent**: When it is set of true, the app is built under the status bar.
  - **barStyle**: It sets the color of status bar text.
  - **hidden**: It is used to hide and show the status bar. By default, it is false. If hidden = {false} it is visible, if hidden = {true}, it hide the status bar.

```
import React from 'react';
import {StatusBar} from 'react-native'
export default function MyStatusBar() {
    return (
      <View style={styles.container}>
        <StatusBar
            backgroundColor = "#b3e6ff"
            barStyle = "dark-content"
            hidden = {false}
            translucent = {true}
        />
      </View>
    );
}
```

Exercise:1: Reusable React-native Component

- Create **your own react native reusable component** using the following core components (**minimum 2**)
  - Text , TextInput
  - Switch, Touchable
  - Button, Image, ImageBackground
  - FlatList, SectionList
  - RefreshControl, Picker, . . .
- A **single stateless/state-full component**
- Use **at least 4 properties (prop)**

- Criteria
  - **Type of component**
  - **Selected properties**
  - **Combination of components**

Thank You

Questions?